

Homework 2: Ensemble Learning for Imbalanced Classification

COVID-19 Severity Prediction Dataset

Field	Details
Deadline	1405 / 02 / 23
Submission	One .zip file named Firstname_Lastname_StudentID.zip
Questions	Post on the course forum or email the TA

Submission Instructions

Please read the following rules carefully. Non-compliant submissions may receive a grade penalty.

What to Submit

- A single **.zip** archive named exactly: `Firstname_Lastname_StudentID.zip`
- The zip must contain **two items**:
 - `code/` — a folder with all your Python source files (one `.py` per task, clearly named, e.g. `task1_hddt.py`, `task2_bagging.py`, `task3_adaboost.py`)
 - `report.pdf` — a single PDF report (details below)

Code Requirements

- **Language:** Python 3.8+
- **Allowed libraries:** NumPy, Pandas, Matplotlib, Seaborn, scikit-learn (only for preprocessing and metrics — NOT for the core classifiers you implement)
- **Each script must be self-contained and executable:** `python task1_hddt.py` should run end-to-end and print all results
- Include a `requirements.txt` if you use any extra packages
- Hardcode the dataset path as `'Covid.csv'` (same folder as the script), or read it from a command-line argument
- Set random seeds for reproducibility (e.g., `np.random.seed(42)`)

Report Requirements

- PDF format, max 15 pages (excluding cover and references)
- Must include: your name, student ID, and section header for each task
- For each experiment: brief method description, result tables/plots, and discussion
- Answer all Theory Questions at the end of the report
- Figures must have captions; tables must have titles

Academic Integrity: All code must be your own. You may discuss ideas with classmates but may NOT share code. Plagiarism will result in a zero grade for the entire homework.

1. Dataset: COVID-19 Severity

You are provided with a real clinical dataset — **Covid.csv** — collected from COVID-19 patients. The task is to predict patient severity (Label) from 40 clinical and demographic features.

1.1 Dataset Overview

Property	Value
Samples	1,585
Features	40 (clinical symptoms + comorbidities + demographics)
Target column	Label → +1 (severe/minority) -1 (non-severe/majority)
Class distribution	Severe (+1): 100 Non-severe (-1): 1,485
Imbalance ratio	~15:1 (this is the core challenge of the homework)
Missing values	Yes — several features have significant missingness (up to 82%)

1.2 Feature Groups

Features fall into four groups. Understanding them will help you interpret model results:

- **Demographics:** age, gender
- **Symptoms:** Fever, Cough, Muscular pain, Shortness of breath, loss of smell/taste, headache, dizziness, nausea, vomiting, diarrhea, etc.
- **Comorbidities:** Diabetes, Cancer, Heart disease, HIV/AIDS, Chronic kidney/liver/lung disease, etc.
- **Clinical indicators:** Intubation, PO2 (blood oxygen), contact with patient

1.3 Preprocessing (Required)

You must perform the following preprocessing steps **before** training any model. Document each step in your report.

1. **Missing value handling:** For each column, report the percentage of missing values. Use an appropriate imputation strategy (mode for binary/categorical features; median for numerical). Justify your choice.
2. **Feature selection via Kendall correlation:** Compute the Kendall rank correlation matrix among all features. Remove one feature from each pair where $|\tau| > 0.5$, keeping the one with higher correlation to the label. Report how many features remain.
3. **Train/test split:** Use a stratified 70/30 split. Stratification is essential given the severe imbalance. Fix `random_state=42` for reproducibility.
4. **Scaling (if needed):** Apply StandardScaler to continuous features only (fit on train, transform on test).

Hint: Use pandas `.isnull().mean()` to get missing rates per column. For Kendall correlation, pandas `.corr(method='kendall')` works but may be slow on 40 features — consider computing only on training data.

Note on leakage: Fit all preprocessing transformers (imputer, scaler) on the TRAINING set only, then apply to the test set. This is a common mistake that inflates results.

2. Task 1 — Hellinger Distance Decision Tree (HDDT)

In this task you will implement the Hellinger Distance Decision Tree from scratch and evaluate it on the binary classification problem.

2.1 Background: Hellinger Distance

The Hellinger distance between two probability distributions P and Q is defined as:

$$H(P, Q) = (1/\sqrt{2}) \cdot \sqrt{\sum_i (\sqrt{p_i} - \sqrt{q_i})^2}$$

For a binary split at node v on feature f , with P = proportion of positive class and Q = proportion of negative class, the Hellinger score for a candidate split into left (L) and right (R) partitions is:

$$H_f = (\sqrt{(p_L/P)} - \sqrt{(n_L/N)})^2 + (\sqrt{(p_R/P)} - \sqrt{(n_R/N)})^2$$

where p_L, n_L = positive/negative counts in left child; p_R, n_R = in right child; P, N = total positives/negatives at node.

Key property: Because each term is normalized by the total class count (P or N) rather than the node size, Hellinger distance is class-prior invariant — it does not favor splits that isolate the majority class, making it well-suited for imbalanced data.

2.2 Implementation

File: `task1_hddt.py`

Implement the HDDT class with the following interface. You may add helper methods:

```
class HDDT:
    def __init__(self, min_samples_split=10, max_depth=None):
        ...

    def fit(self, X, y):          # y in {-1, +1}
        ...

    def predict(self, X):
        ...

    def predict_proba(self, X):  # needed for AUC computation
        ...
```

Your implementation must:

- **Split criterion:** Select the feature and threshold that maximize H_f . For continuous features, evaluate all midpoints between consecutive sorted unique values.
- **Stopping rules:** Stop if (a) node size < `min_samples_split`, (b) all samples have the same label, or (c) `max_depth` is reached.

- **Leaf prediction:** Majority class at the leaf. `predict_proba` returns the fraction of positive samples at the leaf.
- **Data structure:** Use a recursive node/tree structure (class or dict). Do NOT use scikit-learn's `DecisionTreeClassifier` internals.

2.3 Evaluation

Run 10 independent experiments with different random seeds (seeds 0–9) for the train/test split. In each run, train HDDT and record all metrics. Report the mean and standard deviation across the 10 runs.

Metrics to report:

- **Accuracy** (overall)
- **Precision, Recall, F1-score** (for the minority class, label = +1)
- **AUC-ROC** (area under the ROC curve for the minority class)
- **G-mean** = $\sqrt{\text{Sensitivity} \times \text{Specificity}}$

Report results in a table with the following format:

Metric	Mean (10 runs)	Std Dev
Accuracy	...	...
Precision (+1)	...	...
Recall (+1)	...	...
F1-score (+1)	...	...
AUC-ROC	...	...
G-mean	...	...

2.4 Pruning Experiment

Repeat the above evaluation (10 runs each) for the following `max_depth` values: **{None, 2, 3, 4, 5}**. Plot G-mean and F1 vs. `max_depth`. Discuss which depth gives the best bias-variance tradeoff.

Hint: `max_depth=None` means no limit (fully grown). You should see G-mean peak at a moderate depth and then decrease as the tree overfits.

3. Task 2 — Bagging with Undersampling

Bagging (Bootstrap Aggregating) trains multiple classifiers on different bootstrap samples and combines their predictions by majority vote. When combined with undersampling, it is an effective strategy for imbalanced data.

3.1 Algorithm

Bagging with Undersampling — one iteration:

5. Bootstrap sample the **minority class** with replacement (sample size = $|\text{minority}|$).
6. Randomly undersample the **majority class** without replacement to match the minority size.
7. Combine: balanced training subset of size $2 \times |\text{minority}|$.
8. Train a base classifier on this balanced subset.
9. Repeat T times. Final prediction = majority vote over all T classifiers.

Why this works: Each balanced subset exposes the base learner to all minority-class samples and a different random subset of majority-class samples — reducing majority bias while maintaining diversity among base classifiers.

3.2 Implementation

File: task2_bagging.py

Implement the BaggingImbalanced class. You must support two choices of base classifier:

```
class BaggingImbalanced:
    def __init__(self, base_learner='hddt', T=31,
                 max_depth=3, min_samples_split=10):
        # base_learner: 'hddt' (your Task 1 implementation)
        #                 'dt'   (scikit-learn DecisionTreeClassifier)
        ...

    def fit(self, X, y):
        ...

    def predict(self, X):
        ...

    def predict_proba(self, X): # average probabilities across trees
        ...
```

3.3 Experiment: Effect of Ensemble Size T

Use your HDDT as the base learner (max_depth=3). Test ensemble sizes $T \in \{11, 31, 51, 101\}$. For each T , run 10 experiments (seeds 0–9) and report mean $\pm$ std of all metrics.

Report results in this format for each T :

T	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	AUC-ROC	G-mean
11	...	...	...	...	...	...
31	...	...	...	...	...	...
51	...	...	...	...	...	...
101	...	...	...	...	...	...

Plot G-mean and F1 vs. T . At what point does increasing T stop helping?

3.4 HDDT vs. Standard DT as Base Learner

Fix $T = 51$. Compare BaggingImbalanced using:

- Base learner = your HDDT (max_depth=3)
- Base learner = scikit-learn's DecisionTreeClassifier (max_depth=3)

Base Learner	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	G-mean
HDDT (yours)	...	...	...	...	...
Standard DT (sklearn)	...	...	...	...	...

Discuss: does the choice of base learner significantly affect ensemble performance on this heavily imbalanced dataset? Why or why not?

Hint: The G-mean metric is often more revealing than accuracy for imbalanced tasks. If your model predicts everything as -1 it gets >93% accuracy but G-mean = 0.

4. Task 3 — AdaBoost for Imbalanced Data

In this task you implement AdaBoost.M1 and investigate how SMOTE oversampling interacts with boosting for the imbalanced COVID dataset.

4.1 AdaBoost.M1 Implementation

File: `task3_adaboost.py`

Implement AdaBoost.M1 using decision stumps (depth-1 trees) as weak learners. You may use sklearn's `DecisionTreeClassifier(max_depth=1)` as the stump. The boosting logic must be yours.

```
class AdaBoostM1:
    def __init__(self, T=50, learning_rate=1.0):
        ...

    def fit(self, X, y):  # y in {-1, +1}
        # 1. Initialize weights w_i = 1/N
        # 2. For t = 1..T:
        #     a. Train weak learner h_t on weighted data
        #     b. Compute weighted error: eps_t = sum(w_i * 1[h_t(x_i) != y_i])
        #     c. If eps_t >= 0.5: stop (or reset weights)
        #     d. alpha_t = 0.5 * log((1-eps_t) / eps_t)
        #     e. Update weights: w_i *= exp(-alpha_t * y_i * h_t(x_i))
        #     f. Normalize weights
        ...

    def predict(self, X):  # sign of sum(alpha_t * h_t(x))
        ...
```

4.2 SMOTE Implementation

Implement SMOTE (Synthetic Minority Oversampling Technique) from scratch:

10. **Find k nearest neighbors** (k=5) for each minority sample using Euclidean distance.
11. For each minority sample, randomly select one of its k neighbors.
12. Generate a synthetic sample: $x_{\text{new}} = x_i + \lambda * (x_{\text{neighbor}} - x_i)$, where $\lambda \sim \text{Uniform}(0,1)$.
13. Repeat until the minority class reaches the desired size (match majority count).

Important: Apply SMOTE only to the TRAINING set, after splitting. Never apply it before the split or on test data — this would be data leakage and artificially inflate your results.

4.3 Experiments

Experiment A: AdaBoost vs. AdaBoost + SMOTE

Use 10-run stratified evaluation (seeds 0–9). Compare:

- AdaBoost (T=50) on the original imbalanced training data
- AdaBoost (T=50) on SMOTE-oversampled training data

Method	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	AUC-ROC	G-mean
AdaBoost (no SMOTE)	...	...	...	...	...	...
AdaBoost + SMOTE	...	...	...	...	...	...

Experiment B: Number of Boosting Rounds T

With SMOTE applied, vary $T \in \{10, 25, 50, 100\}$. Plot train error, test error, and F1 vs. T. Observe signs of overfitting.

4.4 Full Comparison Across All Methods

In your report, include a final summary table comparing the best configuration of each method:

Method	Best Config	F1 (+1)	G-mean	AUC-ROC
HDDT (Task 1)	max_depth=?	...	...	...
Bagging + HDDT (Task 2)	T=?	...	...	...
Bagging + DT (Task 2)	T=?	...	...	...
AdaBoost (Task 3)	T=?	...	...	...
AdaBoost + SMOTE (Task 3)	T=?	...	...	...

Write a paragraph discussing which method performs best on this dataset and why. Consider the role of class imbalance, ensemble diversity, and data augmentation.

5. Theory Questions

Answer the following questions in your report. Answers should be concise but complete — typically 5–15 lines each. Formal proofs are not required, but reasoning must be clear and supported by evidence from your experiments where applicable.

Q1. Hellinger Distance vs. Gini vs. Information Gain (20 pts)

- (a) Write the formula for Gini impurity and information gain for a binary split and compare them structurally to the Hellinger distance formula given in this homework.
- (b) Explain why Gini and information gain can be biased toward the majority class in imbalanced settings, while Hellinger distance is not. Refer to the normalization terms in the formula.
- (c) Based on your Task 1 results: did HDDT achieve notably better recall for the minority class compared to what you would expect from a standard decision tree (sklearn's default)? Discuss.

Q2. Stable, Unstable, and Weak Classifiers (15 pts)

- (a) Define the concepts of stable and unstable classifiers. Give one concrete example of each.
- (b) Why does Bagging provide the greatest benefit when the base classifier is unstable? Explain in terms of bias-variance decomposition.
- (c) Define a weak classifier. Why does AdaBoost require weak learners rather than strong ones? What happens if you use a fully grown deep tree in AdaBoost?

Q3. Why Max Depth Must Be Limited in Bagging (15 pts)

- (a) In your Task 2 Bagging implementation, you use `max_depth=3` for the base HDDT. Why not use an unlimited depth?
- (b) Explain the concept of diversity among ensemble members. How does bootstrap sampling create diversity? How does undersampling add to this diversity?
- (c) If all base classifiers were identical (zero diversity), what would the ensemble's error equal? Show your reasoning.

Q4. SMOTE and Data Leakage (15 pts)

- (a) What is data leakage and why is it a problem in machine learning pipelines?
- (b) Explain concretely why applying SMOTE before the train/test split constitutes leakage, even though SMOTE only creates synthetic samples from existing minority instances.
- (c) Your experiment in Task 3 should apply SMOTE only to training data. If you accidentally applied it before splitting, how would you expect your reported AUC-ROC to change? Higher or lower than the true value? Why?

Q5. Reflection — Dataset-Specific Challenges (10 pts)

The COVID dataset has ~82% missing values in some columns.

- (a) Which imputation strategy did you choose and why?
- (b) Could the missing values be informative (i.e., the fact that a value is missing carries signal about the label)? Propose a simple way to exploit this.
- (c) In a real clinical deployment, what additional concerns (beyond accuracy) would you have about deploying your best model?

6. Grading Rubric

Component	Points	Notes
Task 1 — HDDT implementation	20	Correct formula, splitting, leaf prediction
Task 1 — Evaluation & pruning experiment	10	10-run avg, all metrics, pruning plot
Task 2 — Bagging implementation	15	Correct bootstrap + undersampling logic
Task 2 — Experiments (T, base learner)	10	Results table + discussion
Task 3 — AdaBoost implementation	15	Correct weight update and alpha computation
Task 3 — SMOTE + experiments	10	No leakage, correct comparison
Theory Questions	15	Q1–Q5 (see per-question pts above)
Report quality	5	Clarity, figures, captions, discussion
Total	100	

Late policy: 10% deduction per day late, up to 3 days. Submissions more than 3 days late will not be graded without prior approval from the TA.